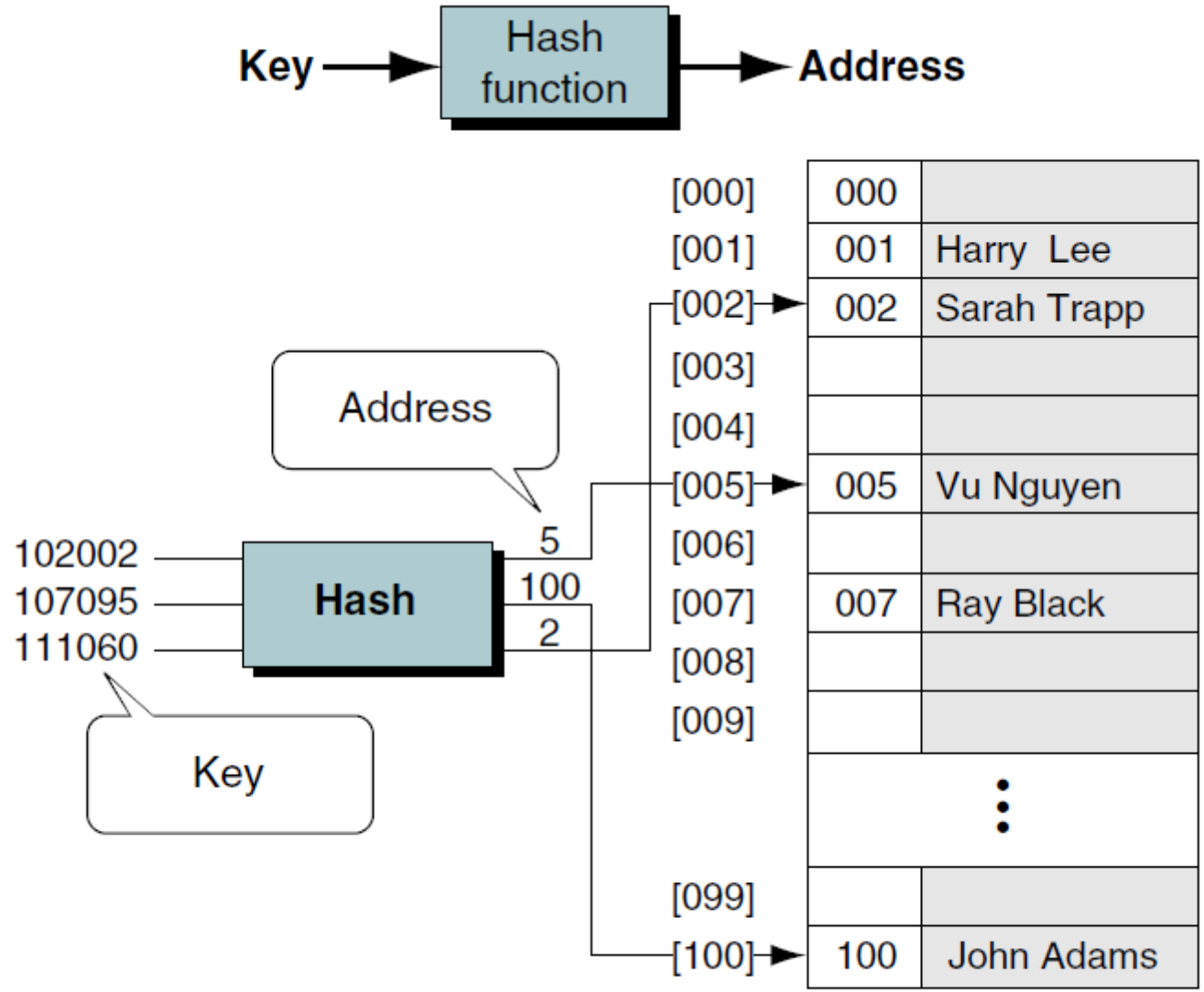
Hash Table

Concept

- In an ideal search, we would know exactly where the data are and go directly there. This is the goal of a hashed search: to find the data with only **one test**.
- In a hashed search, the **key**, through an **algorithmic function**, **determines** the **location of the data**.
- Because we are searching an array, we use a hashing algorithm to **transform the key into the index that contains the data** we need to locate.
- Another way to describe hashing is as a **key-to-address transformation** in which the **keys map** to **addresses** in a **list**.



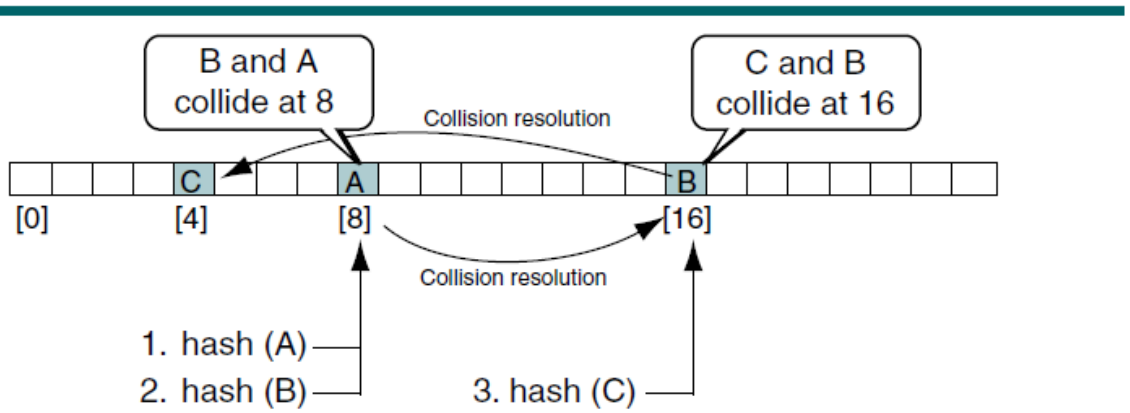
Hash Concept

Concept

- Generally, the **population of keys for a hashed list is greater than the storage area for the data.**
- For example, if we have an array of 50 students for a class in which the students are identified by the last four digits of their Social Security numbers, there are 200 possible keys for each element in the array (10,000 / 50).
- Because there are many keys for each index location in the array, more than one student may hash to the same location in the array. We call the set of keys that hash to the same location in our list **synonyms**.

Concept

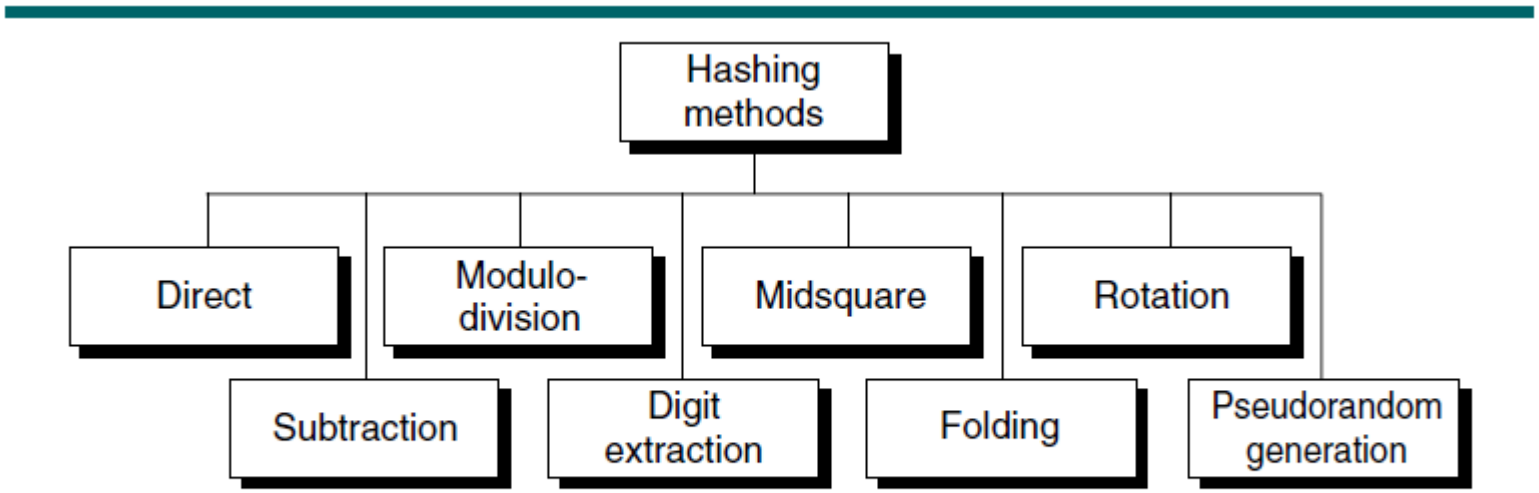
- If the actual data that we insert into our list contain two or more synonyms, we can have **collisions.**
- A collision occurs when a hashing algorithm produces an address for an insertion key and that address is already occupied.
- We must resolve the collision by placing one of the keys and its data in another location.



Collision Resolution Concept

Hashing Methods

- All hash functions except direct hashing and subtraction hashing are many-to-one functions: many keys hash to one address.

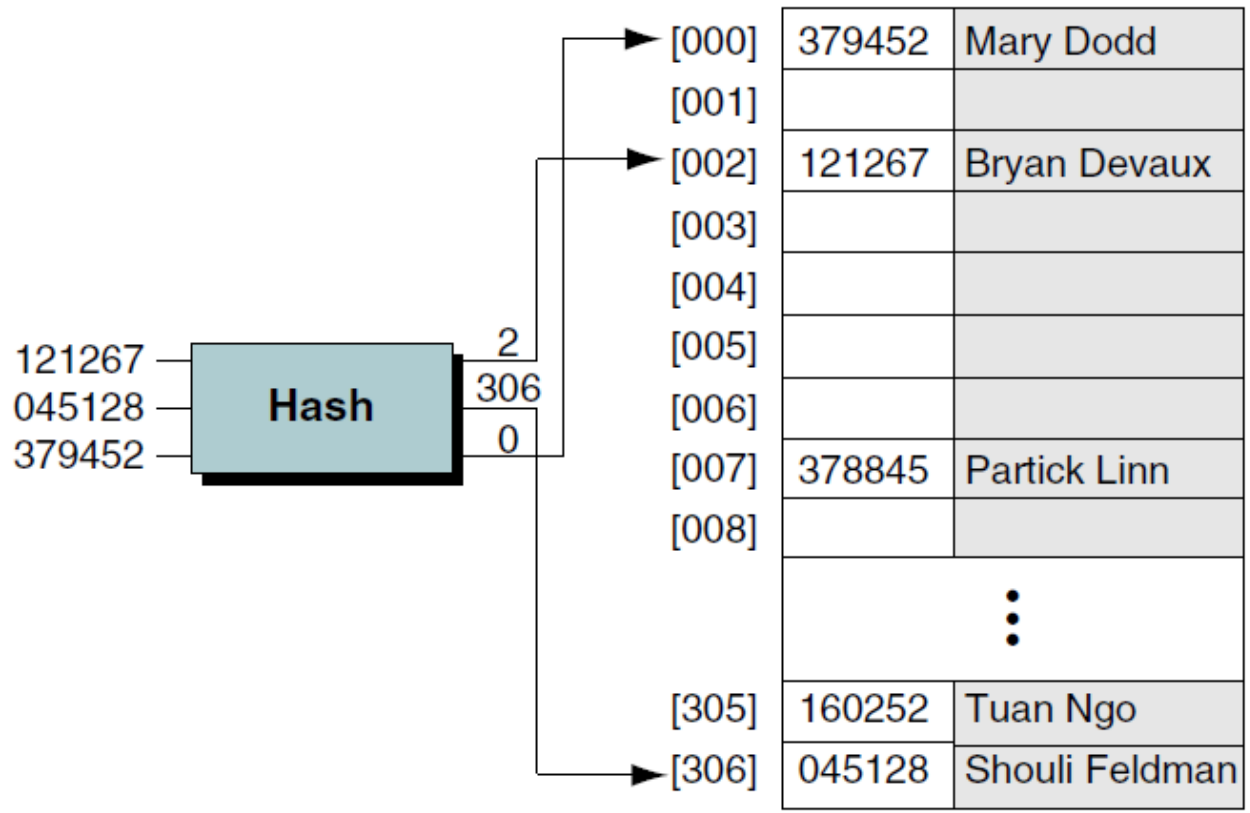


Basic Hashing Techniques

Modulo-division Method

- Formula: ***address = key MODULO listSize***
- A list size that is a prime number produces fewer collisions than other list sizes.
- Primes don't have small divisors. This avoids patterns that happen when the list size shares common factors with the key values.

As our little company begins to grow, we realize that soon we will have more than 100 employees. Planning for the future, we create a new employee numbering system that can handle employee numbers up to 1,000,000. We also decide that we want to provide data space for up to 300 employees. The first prime number greater than 300 is 307. We therefore choose 307 as our list (array) size, which gives us a list with addresses that range from 0 through 306. Our new employee list and some of its hashed addresses are shown in Figure 13-10.



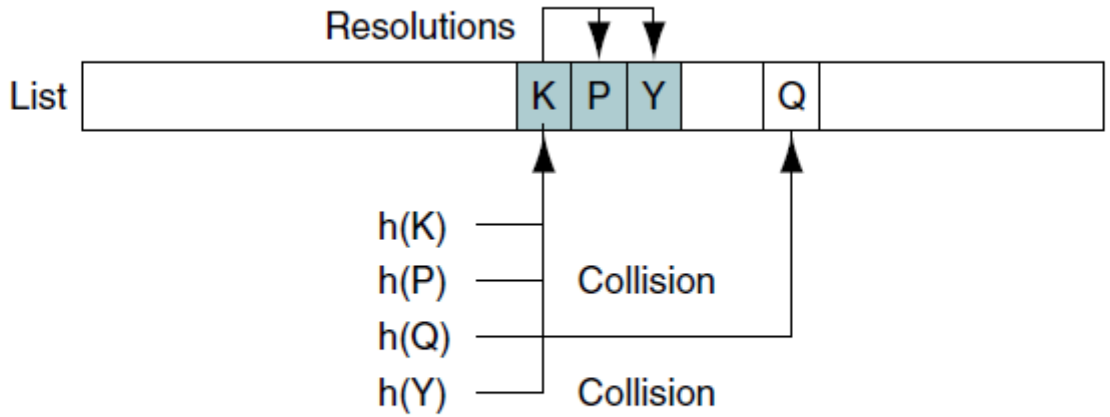
Modulo-division Hashing

Load Factor

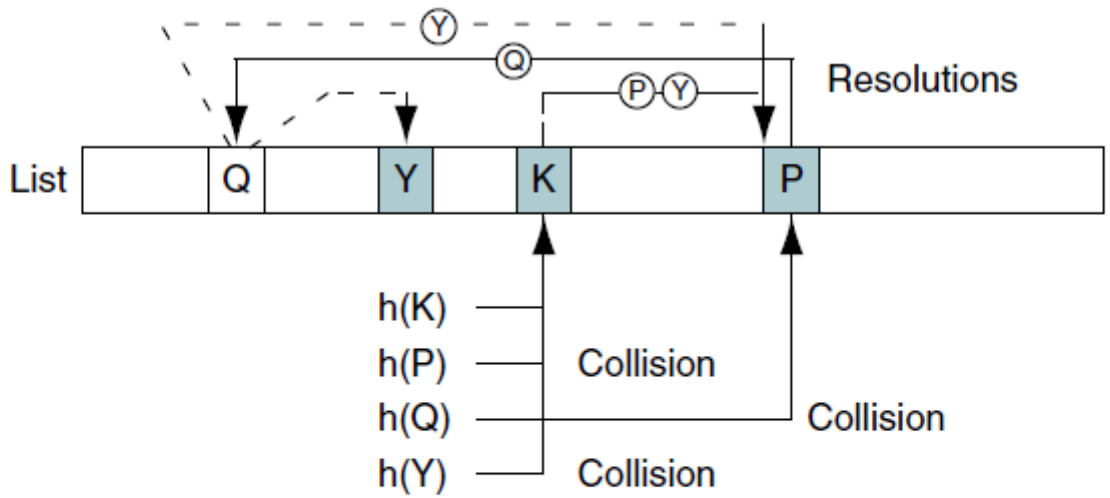
- The load factor of a hashed list is the number of elements in the list divided by the number of physical elements allocated for the list, expressed as a percentage.
- Load factor is assigned the symbol alpha, $\alpha = (k/n) \times 100$
- As a rule of thumb, a hashed list should not become more than 75% full.

Clustering

- Some hashing algorithms tend to cause data to group within the list. This tendency of data to build up unevenly across a hashed list is known as **clustering**.
- Clustering is usually created by collisions.
- If the list contains a high degree of clustering, the number of probes to locate an element grows and reduces the processing efficiency of the list.

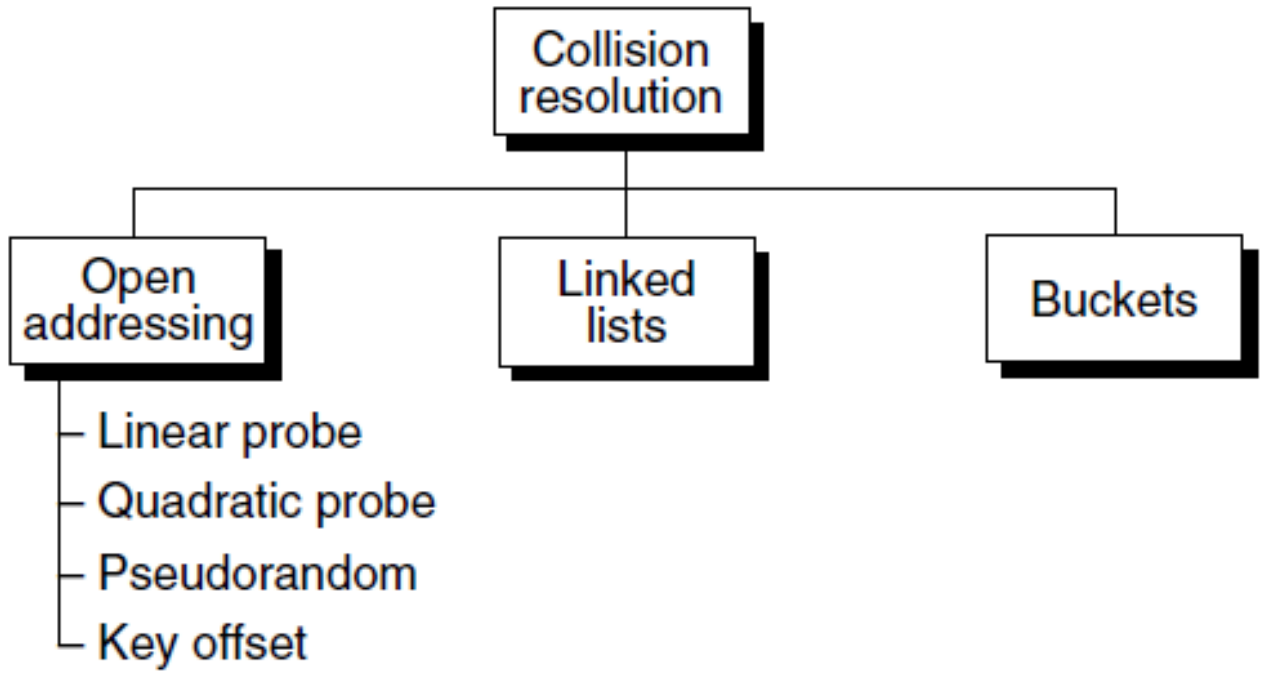


(a) Primary clustering

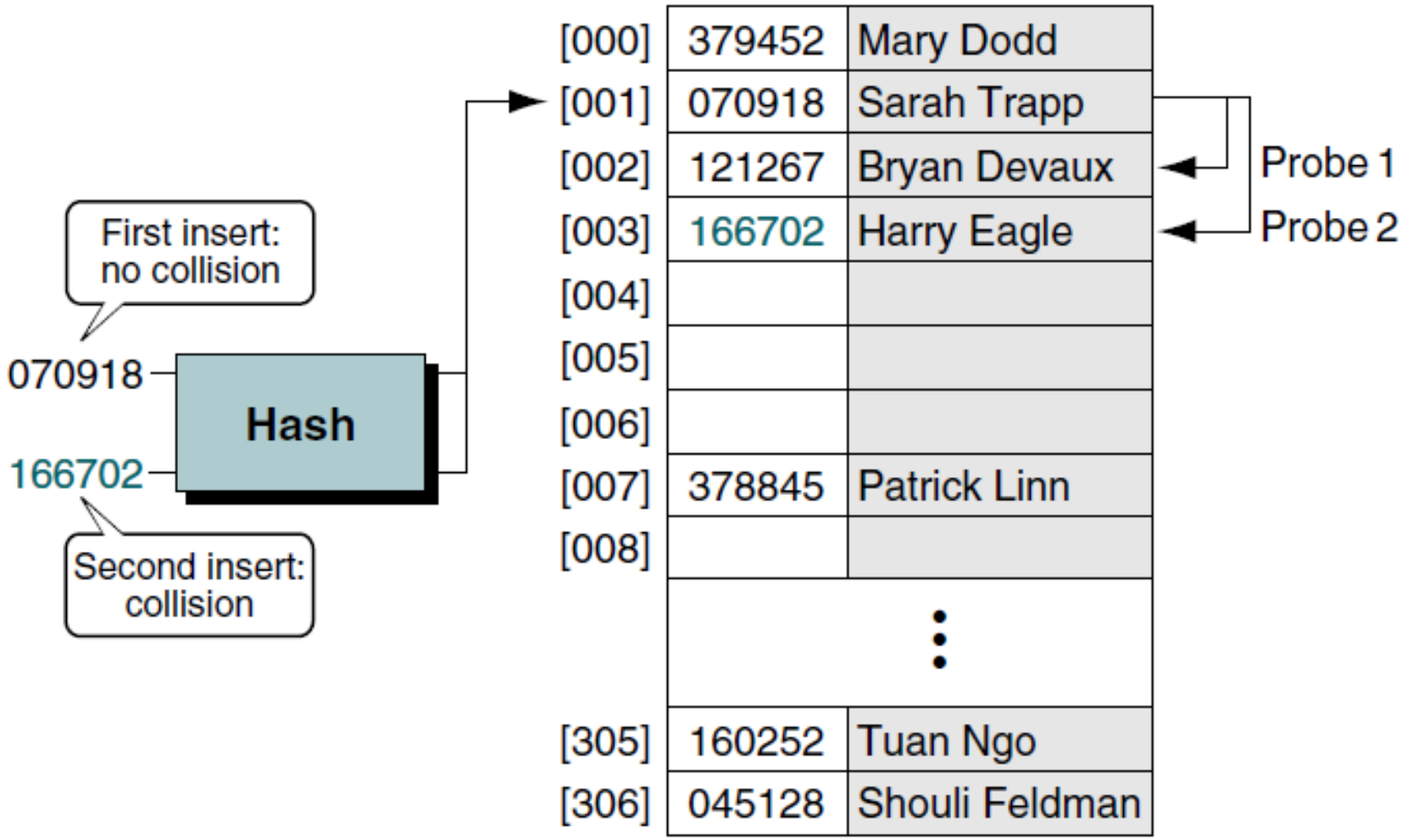


(b) Secondary clustering

One of the surprises of hashing methods is how few elements need to be inserted into a list before a collision occurs. This concept is easier to understand if we recall a common party game used to encourage mingling. The host prepares a list of topics, and each guest is required to find one or more guests who satisfy each topic. One topic is often birthdays and requires that the guests find two people who have the same birthday (the year is not counted). If there are more than 23 party guests, chances are better than 50% that two of them have the same birthday.[5] Extrapolating this phenomenon to our hashing algorithms, if we have a list with 365 addresses, we can expect to get a collision within the first 23 inserts more than 50% of the time.



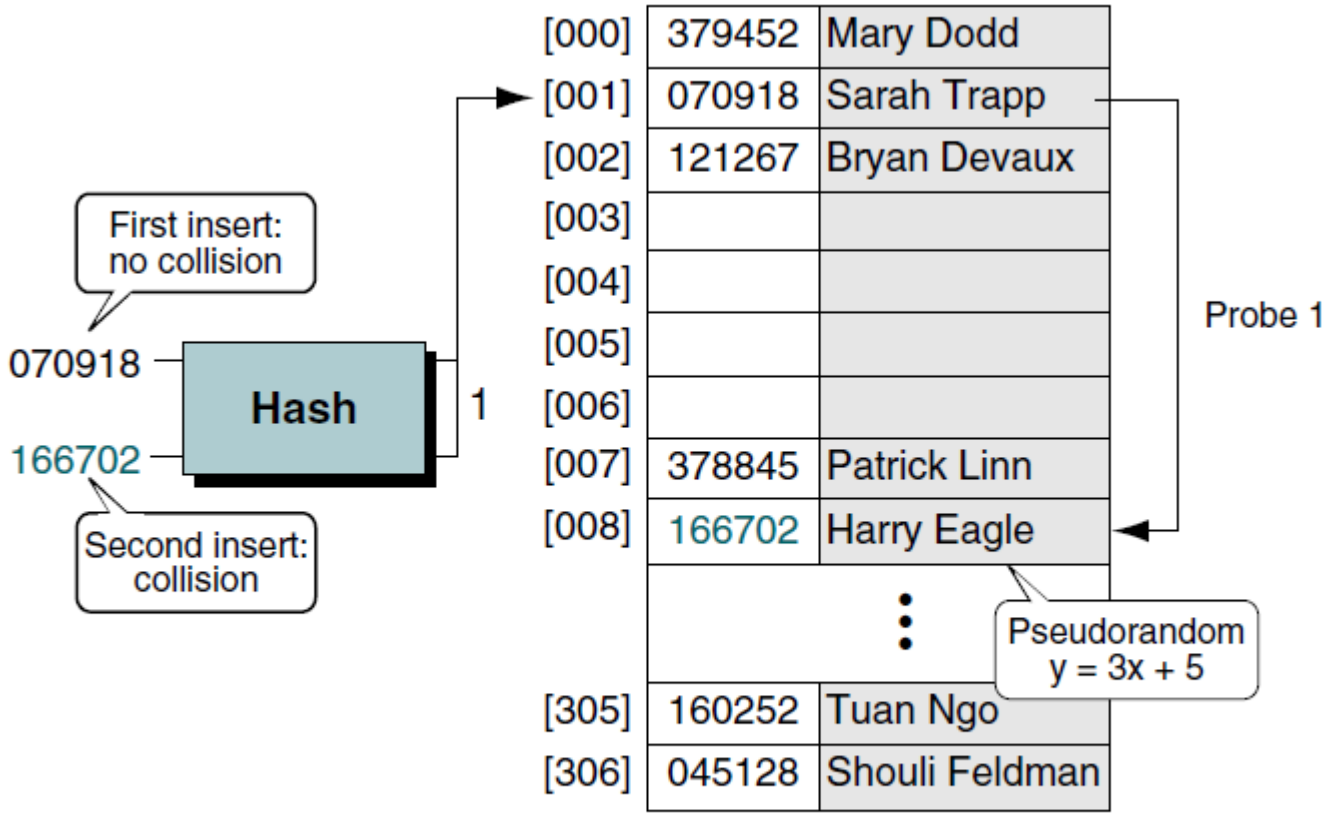
Collision Resolution Methods



Linear Probe Collision Resolution

Probe number	Collision location	Probe^2 and increment	New address
1	1	$1^2 =$ 1	1 + 1 ⇨ 02
2	2	$2^2 =$ 4	2 + 4 ⇨ 06
3	6	$3^2 =$ 9	6 + 9 ⇨ 15
4	15	$4^2 =$ 16	15 + 16 ⇨ 31
5	31	$5^2 =$ 25	31 + 25 ⇨ 56
6	56	$6^2 =$ 36	56 + 36 ⇨ 92
7	92	$7^2 =$ 49	92 + 49 ⇨ 41
8	41	$8^2 =$ 64	41 + 64 ⇨ 05
9	5	$9^2 =$ 81	5 + 81 ⇨ 86
10	86	$10^2 =$ 100	86 + 100 ⇨ 86

Quadratic Collision Resolution Increments



Pseudorandom Collision Resolution

```
y = (ax + c) modulo listSize
  = (3 × 1 + 5) Modulo 307
  = 8
```

Key Offset: A double hashing method

```
offSet  = ⌊key/listSize⌋
address = ((offSet + old address) modulo listSize)
```

For example, when the key is 166702 and the list size is 307, using the modulo-division hashing method generates an address of 1. As shown in Figure 13-16, this synonym of 070918 produces a collision at address 1. Using key offset to calculate the next address, we get 237, as shown below.

```
offSet  = ⌊166702/307⌋ = 543
address = ((543 + 001) modulo 307) = 237
```

If 237 were also a collision, we would repeat the process to locate the next address, as shown below.

```
offSet  = ⌊166702/307⌋ = 543
address = ((543 + 237) modulo 307) = 166
```

Key	Home address	Key offset	Probe 1	Probe 2
166702	1	543	237	166
572556	1	1865	024	047
067234	1	219	220	132

Key-offset Examples